



**Faculty of Mathematics
and Information Sciences**

WARSAW UNIVERSITY OF TECHNOLOGY

SHAP Multi-Modal LLM Explainer

Deployment Documentation, User Manual, and Acceptance Tests

Jakub Muszyński

Paweł Pozorski

January 14, 2026

This document describes the system architecture, deployment procedures, operating instructions, acceptance tests, and project experience summary.

Contents

1	Introduction	3
1.1	Purpose and scope	3
1.2	Document structure	3
2	Requirements and compliance	3
2.1	Functional requirements: front-facing application	3
2.2	Non-functional requirements	4
2.3	Compliance summary	4
3	System architecture and design	4
3.1	High-level architecture	4
3.1.1	Frontend	5
3.1.2	Backend	6
3.1.3	Persistence layer	7
3.2	Sequence diagrams	7
3.2.1	Model loading	7
3.2.2	Attribution computation	8
4	Deployment and operations	8
4.1	Target environment	8
4.2	Configuration	8
4.2.1	Frontend	8
4.2.2	Backend	8
4.3	Production installation (Docker Compose)	9
4.3.1	CPU deployment	9
4.3.2	GPU deployment (optional)	9
5	User manual	9
5.1	Typical workflow	9
5.2	Interpretation notes	10
5.3	Error handling	10

6	Acceptance tests	10
6.1	AT-A1 Model loading (FR-A1)	10
6.2	AT-A2 Text input accepted (FR-A2)	11
6.3	AT-A3 Audio upload gating (FR-A2)	11
6.4	AT-A4 Sampling-based attribution with progress and cancellation (FR-A3, NFR-2) . .	11
6.5	AT-A5 Exact SV constraints enforced (FR-A3)	11
6.6	AT-A6 Visualizations render correctly (FR-A5)	12
6.7	AT-A7 Cost estimation updates (FR-A6)	12
6.8	AT-A8 Session persistence and reload (FR-A7, NFR-3)	12
6.9	AT-A9 Export (FR-A8)	12
6.10	AT-A10 Telemetry (FR-A9)	12
7	Project experience and lessons learned	13
7.1	Engineering observations	13
7.2	Practical trade-offs	13
8	Appendix: deployment commands	13
8.1	CPU	13
8.2	GPU (optional)	13

1 Introduction

1.1 Purpose and scope

This document provides a complete, production-oriented description of the SHAP Multi-Modal LLM Explainer application. It covers deployment documentation (environment, configuration, installation), production startup steps, acceptance tests aligned with the initial requirements, a user manual, and a summary of lessons learned during development.

The system is designed for interactive, code-free exploration of Shapley-value attributions for model outputs in text and audio modalities.

1.2 Document structure

The document is organized into requirements and compliance, system architecture and design, deployment and operations, user instructions, acceptance tests, and a project experience summary.

2 Requirements and compliance

2.1 Functional requirements: front-facing application

The following items are addressed by the delivered system:

- **FR-A1 (Model loading):** A GUI workflow loads a supported model connector and exposes basic configuration (source/mode, device, precision). The backend validates and initializes the connector.
- **FR-A2 (Modalities):** The GUI accepts text input and audio upload. Multimodal processing is available in the multimodal connector mode; text-only mode rejects audio.
- **FR-A3 (SV methods):** Exact Shapley values are available for short text-only inputs. For longer inputs the GUI exposes sampling-based approximations, including permutation Monte Carlo and Neyman stratified allocation.
- **FR-A4 (Granularity):** Text explanations are returned at token granularity. Audio explanations are returned as a time-series (segment-like) attribution signal aligned to duration.
- **FR-A5 (Visualization):** The GUI renders token-level overlays for text (heatmap and bar views) and time-aligned intensity plots for audio, plus comparative modality views.
- **FR-A6 (Cost/latency estimation):** The GUI displays an estimate of the number of model evaluations and a rough time budget based on method parameters.
- **FR-A7 (Session history):** Configurations and results are stored locally in a database and can be reloaded for comparison.
- **FR-A8 (Export):** Exports are supported for machine-readable attribution data and publication-ready figures.
- **FR-A9 (Telemetry, optional):** A lightweight telemetry panel shows runtime statistics such as evaluation counts and caching indicators without storing sensitive content by default.

2.2 Non-functional requirements

- **NFR-1 (Usability):** The end-user workflow is GUI-driven and documented, with sensible defaults and minimal required parameters.
- **NFR-2 (Performance):** Long-running explainability computations report progress and can be cancelled; the interface remains responsive during computation.
- **NFR-3 (Reproducibility):** Runs record seed and method parameters; session reload restores configuration and results. Determinism depends on model/runtime but is supported as far as feasible.
- **NFR-4 (Portability):** The system is containerized and runs on a standard Linux environment; GPU support is optional.
- **NFR-5 (Reliability):** Failures in model calls or parsing are surfaced with actionable messages; the system avoids silent discarding of results.
- **NFR-6 (Security and privacy):** No external transmission of user inputs or attributions occurs unless explicitly triggered by the user (e.g., exporting). Model acquisition from public hubs occurs only during explicit model load operations.

2.3 Compliance summary

The application fulfills the functional and non-functional requirements through a GUI-driven workflow backed by a containerized FastAPI service. Exact SV computation is restricted to feasible scenarios, sampling-based methods provide scalability, and session persistence enables reproducible comparison. Privacy is preserved by local processing and explicit export actions.

3 System architecture and design

3.1 High-level architecture

The application consists of a web frontend, a FastAPI backend, and a PostgreSQL database for persisting sessions. Docker Compose is used for local and production-like orchestration.

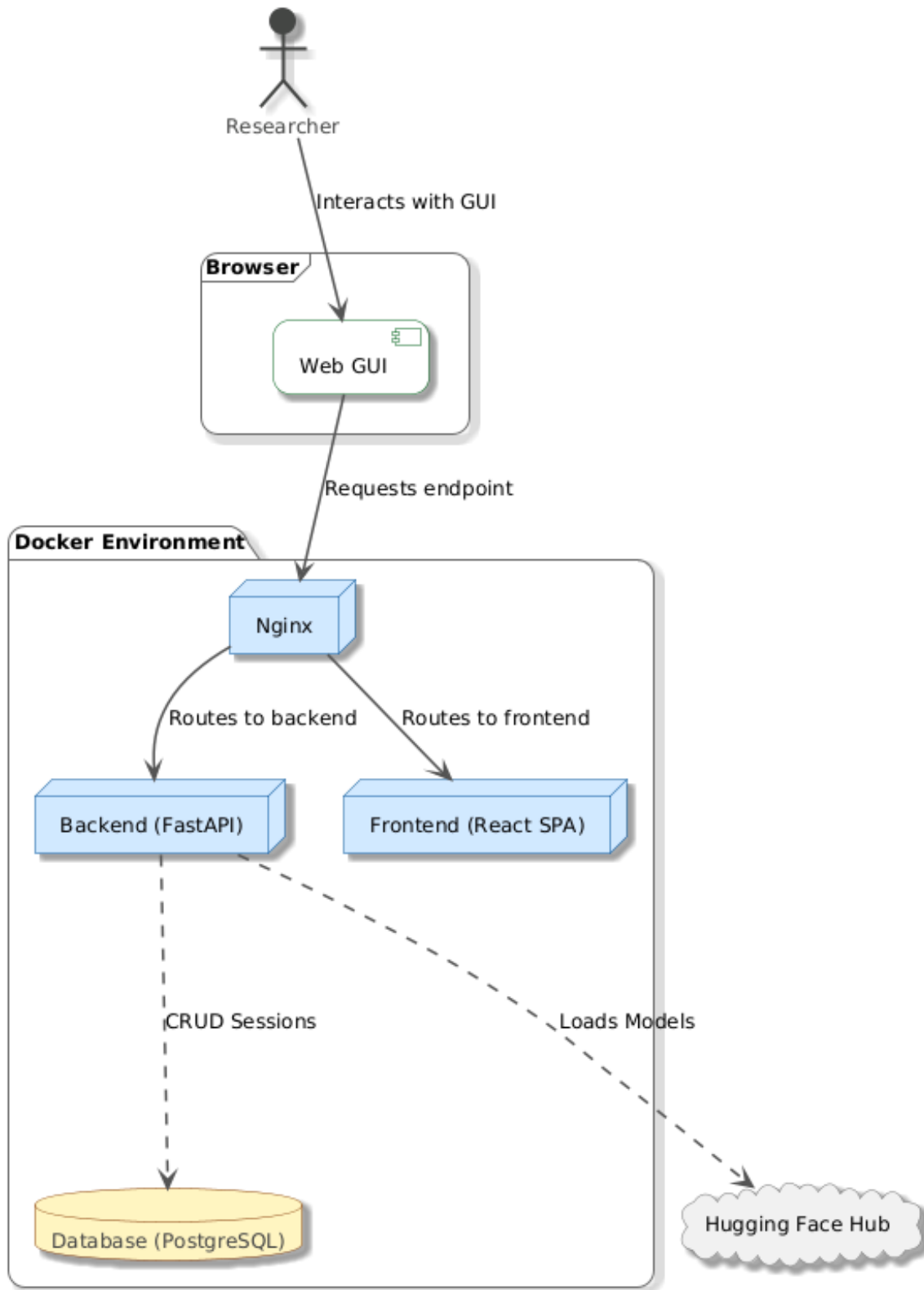


Figure 1: General architecture of the system.

3.1.1 Frontend

The frontend is implemented in React (Vite build) and provides:

- model selection and loading controls
- text and audio input panel
- method configuration for SV estimation
- progress tracking and cancellation controls
- visualization and comparison views
- session history management
- export of figures and attribution data
- optional telemetry display

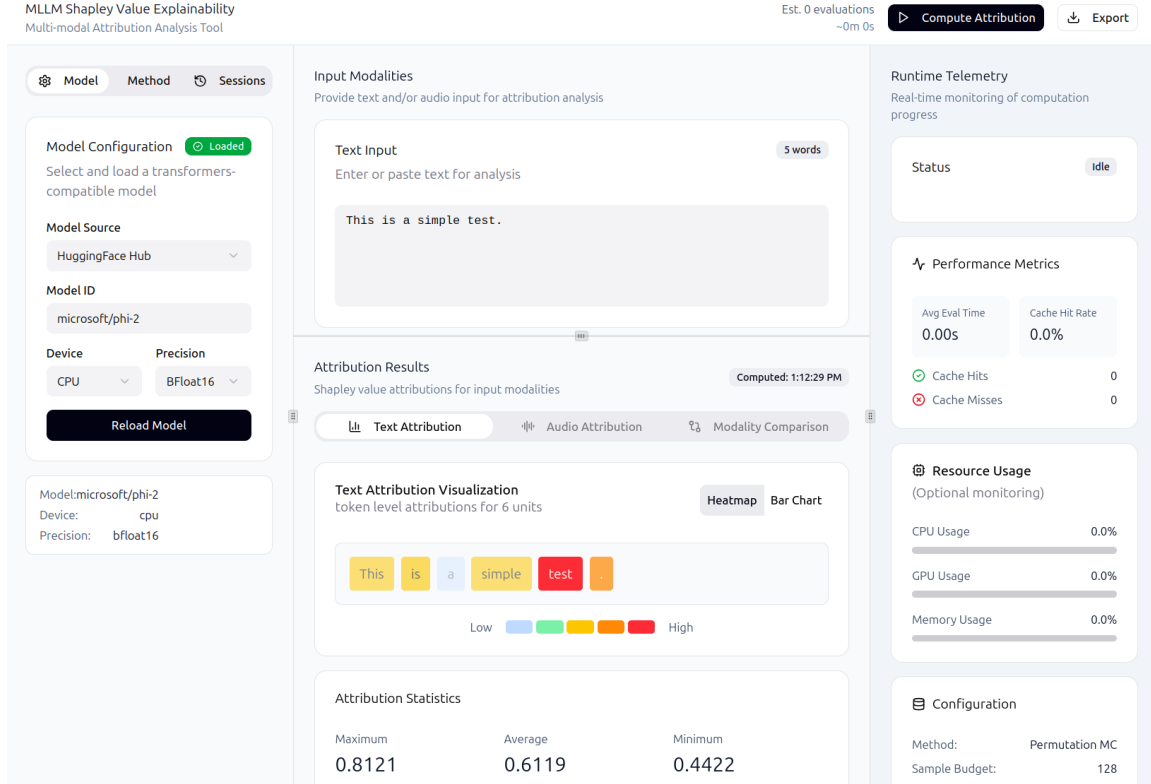


Figure 2: Graphical user interface of the application.

3.1.2 Backend

The backend is implemented in FastAPI and provides endpoints for:

- model loading and status
- prediction
- SV explainability jobs
- progress tracking and cancellation
- per-job logs
- telemetry

Internally, the backend uses an explainer service that runs exact or sampling-based SV computation, and returns token-level text contributions and time-series audio contributions.

3.1.3 Persistence layer

Session metadata, configuration snapshots, and resulting attributions are persisted in a database. The stored sessions can be listed, loaded, and compared from the GUI.

3.2 Sequence diagrams

3.2.1 Model loading

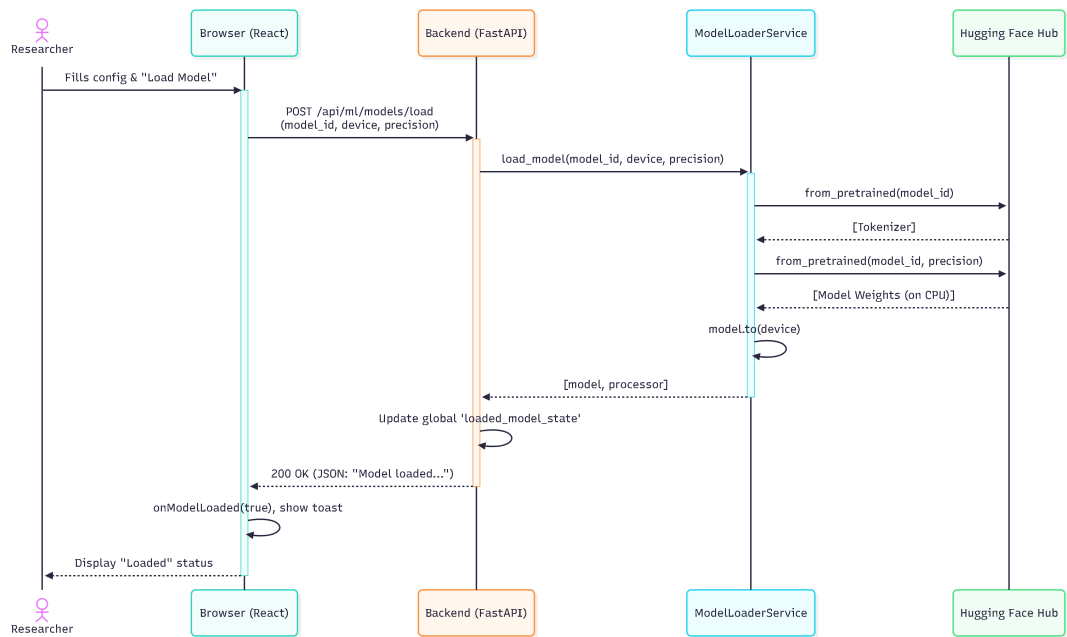


Figure 3: Sequence diagram of model loading.

3.2.2 Attribution computation

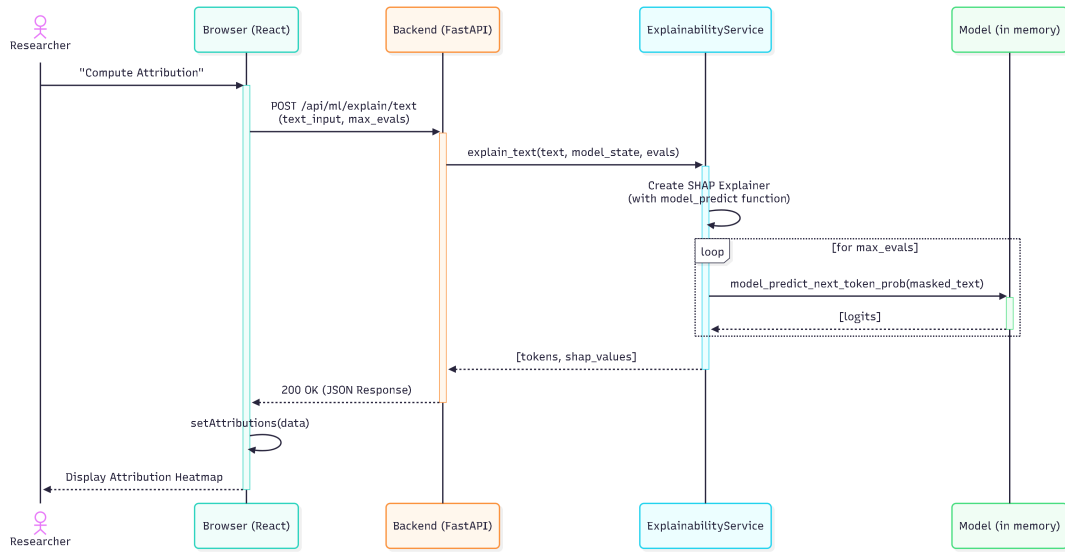


Figure 4: Sequence diagram of computing attributions.

4 Deployment and operations

4.1 Target environment

- **Operating system:** Linux
- **Recommended runtime:** Docker Engine + Docker Compose
- **GPU support (optional):** NVIDIA driver and NVIDIA Container Toolkit

Containerization is used to avoid local dependency drift and to make experiments reproducible.

4.2 Configuration

4.2.1 Frontend

The frontend reads the backend base URL from the environment variable `VITE_API_BASE_URL` (default: `/api`).

4.2.2 Backend

The backend exposes a FastAPI server and uses environment-driven configuration for logging and run-time behavior. The database is initialized on startup and used for session persistence.

4.3 Production installation (Docker Compose)

4.3.1 CPU deployment

1. Build and start the stack:

```
docker compose up --build
```

2. Open the application:

- `http://localhost`

4.3.2 GPU deployment (optional)

Prerequisites on the host:

- NVIDIA driver installed and working (e.g., `nvidia-smi`)
- NVIDIA Container Toolkit installed and configured for Docker

Run:

```
docker compose -f docker-compose.yaml -f docker-compose.gpu.yaml up --build
```

Verify CUDA availability inside the backend container:

```
docker compose exec backend uv run python -c "import torch;
print('cuda_available=', torch.cuda.is_available());
print('torch_cuda=', torch.version.cuda)"
```

5 User manual

5.1 Typical workflow

1. **Load a model**

- Open the *Model* panel.
- Select a connector mode.
- Choose device and precision.
- Load the model.

2. **Provide inputs**

- Enter text into the text field.
- Optionally upload audio (available only for multimodal mode).

3. Select an SV method

- For short text-only inputs, select exact SV.
- For longer inputs, select a sampling-based method.
- Set the sample budget and the random seed.

4. Run attribution

- Start the computation.
- Monitor progress.
- Cancel if necessary.

5. Inspect results

- Use text attribution views (heatmap and bars).
- Use audio attribution views (time-aligned intensity).
- Use modality comparison view for aggregated comparisons.

6. Persist and export

- Save a session.
- Export attribution data (JSON/CSV) and figures (SVG/PNG).

5.2 Interpretation notes

- **Text attributions** represent token-level contributions of the input text to the model output.
- **Audio attributions** represent the contribution of time segments of the audio input to the model output.
- **Comparative views** aggregate contributions by modality using absolute sums and descriptive statistics.

5.3 Error handling

- If no model is loaded, computation is blocked with a clear message.
- If audio is provided while the system is in text-only mode, the request is rejected and the UI indicates why.
- If a computation fails, the failure is reported with actionable information and the progress indicator is stopped.

6 Acceptance tests

The acceptance tests are designed to be executed through the GUI and validated through observable outcomes. Where needed, API logs can be used as supporting evidence.

6.1 AT-A1 Model loading (FR-A1)

Preconditions: The stack is running.

Steps:

-
1. Open the GUI.
 2. Select a connector mode.
 3. Configure device and precision.
 4. Load the model.

Expected result:

- The UI indicates that the model is loaded.
- Subsequent prediction and explanation operations are enabled.

6.2 AT-A2 Text input accepted (FR-A2)

Steps: Enter text in the input field.

Expected result: Word count and input state update immediately.

6.3 AT-A3 Audio upload gating (FR-A2)

Steps:

1. Switch to text-only mode.
2. Observe the audio input section.

Expected result: Audio is disabled and cannot be submitted; backend rejects audio requests in this mode.

6.4 AT-A4 Sampling-based attribution with progress and cancellation (FR-A3, NFR-2)

Steps:

1. Load a multimodal connector.
2. Provide text and optionally audio.
3. Select a sampling method and a non-trivial budget.
4. Start the computation.
5. Cancel during execution.

Expected result:

- Progress updates are visible during execution.
- Cancellation stops the computation and returns the UI to an idle state.

6.5 AT-A5 Exact SV constraints enforced (FR-A3)

Steps:

-
1. Select exact SV.
 2. Provide a long text input and/or audio.
 3. Start explanation.

Expected result: The system rejects unsupported combinations (e.g., audio with exact SV) and provides an explicit message.

6.6 AT-A6 Visualizations render correctly (FR-A5)

Steps: After a successful run, open text and audio attribution views.

Expected result:

- Text overlay (heatmap and bars) renders consistently.
- Audio time-aligned intensity plot renders with an interpretable time axis.

6.7 AT-A7 Cost estimation updates (FR-A6)

Steps: Change the method or the sample budget.

Expected result: Estimated evaluation count and time budget update accordingly.

6.8 AT-A8 Session persistence and reload (FR-A7, NFR-3)

Steps:

1. Run an attribution.
2. Save the session.
3. Reload the session.

Expected result: The configuration and results are restored and remain comparable to the original run.

6.9 AT-A9 Export (FR-A8)

Steps: Export attribution data and a figure from the export dialog.

Expected result: Downloaded artifacts are created successfully and reflect the selected view and data.

6.10 AT-A10 Telemetry (FR-A9)

Steps: During attribution computation, open the telemetry display.

Expected result: Runtime statistics are visible without exposing or storing the full input content by default.

7 Project experience and lessons learned

7.1 Engineering observations

- **Progress and cancellation require cooperative design.** The system must propagate cancellation signals into the sampling loop; a UI control alone is insufficient.
- **Exact SV must be guarded.** Exact computation is only feasible for short inputs; explicit constraints prevent accidental exponential runtimes.
- **Reproducibility benefits from persistence.** Persisting configuration and results makes comparative experimentation practical and reduces error-prone manual bookkeeping.
- **Visualization must be robust to outliers.** Attribution distributions often contain spikes; clipping and adaptive scaling are necessary for readable plots.

7.2 Practical trade-offs

- **Model loading is intentionally constrained to supported connectors.** This prioritizes reliability and comparability of results.
- **Granularity is token-based for text and segment-like for audio.** Dialogue-level aggregation is supported through stored sessions and view comparisons; dedicated per-turn aggregation controls can be added if required by future study designs.

8 Appendix: deployment commands

8.1 CPU

```
docker compose up --build
```

8.2 GPU (optional)

```
docker compose -f docker-compose.yaml -f docker-compose.gpu.yaml up  
--build
```